KUBERNETES

It is an open source container orchestration platform used to Deploy, scale & manage containerized Applications.

- originally Developed at Google and open-sourced in 2014.
- Inspired by google's internal container managment system, Borg.
- CNCF = cloud Native Computing Foundation, which host the kubernetes project.

*short form is
K8S = kubernetes

→ **Software Deployment**

It is the process of making an Application available to users on a server or over the internet.

↳ **Deployment involves**

- configuring servers
- installing Dependencies
- Managing Traffic
- Maintaining Application Availability.

↳ **Main challenge** - Ensuring Applications run reliably across different enviroments & handle increasing traffic.

→ Deployment Methods

↳ (A) Bare Metal Deployment

- Here, Runs Applications Directly on physical servers.
- Requires manual installation of operating systems, runtimes, databases & Dependencies.
- **Vertical scaling** - Increasing CPU, RAM or other resources on an existing server.
- **Limitations** - High Maintanance costs, manual configuration, hardware dependency & slow scaling.

↳ (B) Cloud Deployment

- Here, It uses on-demand Infrastructure provided by cloud platforms.
- Example - AWS, Google cloud, Microsoft Azure
- Supports managed Databases, load balancer, CDN's & Automatic scaling.
- **Limitations** - Enviroment configuration & dependency issues can still occur.

(c) Virtualization

- Technology that allows multiple virtual machines to run on one physical machine.
- Each VM has its own guest operating system, libraries & Applications.
- **Hypervisor** - software that creates & manages virtual machines.
- **Advantages** - Isolation & consistent enviroments.
- **Limitations** - Higher resource usage, large images, & slower startup compared with Containers.

→ **Containerization** - Packing an Application & its dependencies into a container image for consistent image.

Some Terms:

- **containerImage** - Template used to create containers.
- **container** - running used to create containers.
- **containerd** - A runtime that manages container execution and lifecycle.
- **CRI (contained Runtime Interface)** - The interface Kubernetes uses to communicate with container runtime.
- **CRI-O** : A container runtime designed for kubernetes.

→ Container Orchestration

Automating the Deployment, scaling, networking & Management of containers.

- Required when Applications run hundreds or thousands of containers across multiple servers.

↳ Main Functions -

- Automatically scale application workloads.
- Restart or replace failed workloads.
- Monitor Application health.
- Manage networking & service discovery.
- Distribute workloads across available machines.
- Support rolling updates & Application availability.

→ Uses of Kubernetes:

- Automated Deployment & scaling.
- container health management.
- service discovery and load balancing.
- Rolling updates & rollbacks
- Workload scheduling across multiple nodes.
- Recovery from many container & node failures.

→ Kubernetes Architecture

Kubernetes Architecture consists of two main parts.

↳ **(A) Control Plane** - Manages the cluster & makes scheduling & coordination decisions.

- **kube-apiserver** - Entry point for kubernetes API requests ; validates & processes instructions.
- **etcd** - Distributed Key-value store that maintains cluster state & configuration data.
- **kube-schedular** - Assigns Pods to suitable nodes.
- **kube-controller-manager** :- Maintains the desired cluster state.
- **CCM** :- (cloud controller Manager) - Manages cloud resources like load balancers & routes.

↳ **(B) Worker Nodes** - Run the Application's workloads.

- **kubelet** - Ensures assigned Pods are running.
- **kube-proxy** - Manages service network routing.
- **Container Runtime** - Runs container (containerd, CRI-O)
- **Pod** - Smaller deployable unit in Kubernetes.

→ Important Kubernetes Concepts

- **cluster** - A group of Machines managed by kubernetes.
- **Node** - A machine that participates in a kubernetes cluster.
- **Pod** - The smallest deployment unit, containing one or more containers.
- **Deployment** - A kubernetes resource that manages replicated application Pods & supports updata.
- **service** - Provides a stable network endpoint for accessing a group of pods.
- **Replica** - An instance of an application Pod.
- **Namespace** - Logically seperates resources within a cluster.
- **configMap** - stores non-sensitive configuration data.
- **secret** - stores sensitive configuration data, although additional security controls are needed to protect it.

→ Desired state and Reconciliation Loop

↳ **Desired state** - The configuration specifying how the application should run.

↳ **Actual state** - The currently observed condn of the cluster.

↳ **Reconciliation Loop** - The continuous process of comparing actual & desired states & taking corrective action.

→ Virtual Machines Vs Containers

Feature	Virtual Machines	containers
Operating system	separate Guest OS	share host kernal
Resource usage	Generally higher	Generally lower
startup	Generally slower	Generally faster
Isolation	Virtualized hardware & OS	OS-level isolation
Portability	VM Images	Container Images

Kubernetes Architecture

